核心問題診斷

- 影響範圍：`src/editor/extensions.ts` → `SafeImage` / `createSafeImageNodeView`
- 問題類型：邏輯錯誤（事件衝突）

```
- **問題本質**：`<img>` 元素在瀏覽器中預設具有原生 `draggable=true` 行為（拖曳出圖片資源），且
```

Codex 執行指令

2.1 變更定位

- **Target File/Module:** `src/editor/extensions.ts`
- **Target Scope:** `createSafeImageNodeView` 函數（`render` 內建立 `<img>` 的段落、以及新增拖曳手把）

2.2 問題代碼分析

現有邏輯缺陷：

```
const image = document.createElement("img");
image.src = src;
image.alt = alt;
if (title) image.title = title;
mediaWrap.replaceChildren(image, resizeHandle);
```

`image` 沒有設定 `draggable = false`，瀏覽器沿用其原生預設值 `true`，與外層 `figure.draggable = true` 互相競爭，實際生效的拖曳來源會是 `<img>` 本身而非 ProseMirror 認識的節點邊界。

修復規格要求：

1. 建立 `<img>` 時明確 `image.draggable = false`，讓 mousedown 沿 DOM 往上尋找可拖曳祖先時，正確落在 `figure`（ProseMirror node view 的 dom）身上。
2. 新增一個明顯的拖曳把手（六點圖示，比照 Notion），常駐於 hover 工具列旁，`draggable` 不需額外設定（沿用 figure 的拖曳範圍即可），純粹作為視覺提示，滑鼠移到圖片上就能看到「可拖曳」提示，改善可發現性。
3. `resizeHandle`、`toolbar` 內的按鈕、`figcaption` 皆為非圖片主體元素，預設 `draggable` 為 `false`，不需更動，且 `stopEvent` 已正確排除這些元素、並放行所有 `drag*` 事件，不需修改。

2.3 完整修復代碼

`createSafeImageNodeView` 函數（完整取代原函數）：

```
export function createSafeImageNodeView(node: ProseMirrorNode, view: EditorView, getP
  const nodeType = node.type;
  let attributes = node.attrs as SafeImageAttributes;
  let remoteLoaded = false;
  let showCaptionInput = Boolean(attributes.caption);
  let resizing = false;
  let resizeMove: ((event: PointerEvent) => void) | null = null;
  let resizeFinish: (() => void) | null = null;

  const figure = document.createElement("figure");
  figure.className = "safe-image-node";
  figure.setAttribute("contenteditable", "false");
  figure.draggable = true;

  const dragHandle = document.createElement("span");
  dragHandle.className = "image-drag-handle";
  dragHandle.setAttribute("aria-hidden", "true");
  dragHandle.title = t("image.dragHandle");

  const mediaWrap = document.createElement("div");
  mediaWrap.className = "safe-image-media";

  const toolbar = document.createElement("div");
  toolbar.className = "image-toolbar";

  const captionButton = document.createElement("button");
  captionButton.type = "button";
  captionButton.className = "image-toolbar-btn";
  captionButton.title = t("image.toggleCaption");
  captionButton.setAttribute("aria-label", t("image.toggleCaption"));
  captionButton.textContent = t("image.toggleCaptionShort");

  const zoomButton = document.createElement("button");
  zoomButton.type = "button";
  zoomButton.className = "image-toolbar-btn";
  zoomButton.title = t("image.zoom");
  zoomButton.setAttribute("aria-label", t("image.zoom"));
  zoomButton.textContent = t("image.zoomShort");

  const deleteButton = document.createElement("button");
  deleteButton.type = "button";
  deleteButton.className = "image-toolbar-btn image-toolbar-danger";
  deleteButton.title = t("image.delete");
  deleteButton.setAttribute("aria-label", t("image.delete"));
  deleteButton.textContent = t("image.deleteShort");
  toolbar.append(captionButton, zoomButton, deleteButton);

  const resizeHandle = document.createElement("span");
  resizeHandle.className = "image-resize-handle";
  resizeHandle.setAttribute("aria-hidden", "true");

  const figcaption = document.createElement("figcaption");
```

```
  figcaption.contentEditable = "true";
  figcaption.className = "image-caption";
  figcaption.setAttribute("data-placeholder", t("image.captionPlaceholder"));

  const render = () => {
    const source = imageSource(attributes);
    const src = String(attributes.src ?? source);
    const alt = imageAlt(attributes);
    const title = typeof attributes.title === "string" ? attributes.title : "";
    const width = typeof attributes.width === "string" && /^\d+(?:\.\d+)?%$/.test(at
      ? attributes.width
      : null;
    figure.style.width = width ?? "";
    if (isRemoteImageSource(source) && !remoteLoaded) {
      mediaWrap.className = "remote-image-placeholder";
      mediaWrap.setAttribute("role", "button");
      mediaWrap.setAttribute("tabindex", "0");
      mediaWrap.setAttribute("data-remote-src", source);
      mediaWrap.setAttribute("aria-label", t("image.remoteBlockedAria", { source: al
      mediaWrap.replaceChildren(document.createTextNode(t("image.remoteBlocked", { s
      figure.replaceChildren(mediaWrap);
      return;
    }
    mediaWrap.className = "safe-image-media";
    mediaWrap.removeAttribute("role");
    mediaWrap.removeAttribute("tabindex");
    mediaWrap.removeAttribute("data-remote-src");
    mediaWrap.removeAttribute("aria-label");
    const image = document.createElement("img");
    image.src = src;
    image.alt = alt;
    image.draggable = false;
    if (title) image.title = title;
    mediaWrap.replaceChildren(image, dragHandle, resizeHandle);
    figcaption.textContent = typeof attributes.caption === "string" ? attributes.cap
    figure.replaceChildren(mediaWrap, toolbar, ...(showCaptionInput ? [figcaption] :
  };

  const loadRemote = () => {
    if (!isRemoteImageSource(imageSource(attributes)) || remoteLoaded) return;
    remoteLoaded = true;
    render();
  };
  const onClick = (event: MouseEvent) => {
    if ((event.target as HTMLElement).closest(".remote-image-placeholder")) loadRemo
  };
  const onKeyDown = (event: KeyboardEvent) => {
    if (event.key !== "Enter" && event.key !== " ") return;
    event.preventDefault();
    loadRemote();
  };
  mediaWrap.addEventListener("click", onClick);
  mediaWrap.addEventListener("keydown", onKeyDown);

  const onCaptionClick = () => {
    showCaptionInput = !showCaptionInput;
```

```
    render();
    if (showCaptionInput) window.requestAnimationFrame(() => figcaption.focus());
  };
  const onCaptionBlur = () => {
    setImageAttribute(view, getPos, { caption: figcaption.textContent?.trim() || nul
  };
  const onZoomClick = () => {
    const source = imageSource(attributes);
    figure.dispatchEvent(new CustomEvent(IMAGE_ZOOM_REQUESTED_EVENT, {
      bubbles: true,
      detail: { src: String(attributes.src ?? source), alt: imageAlt(attributes) },
    }));
  };
  const onDeleteClick = () => deleteImageNode(view, getPos);
  captionButton.addEventListener("click", onCaptionClick);
  figcaption.addEventListener("blur", onCaptionBlur);
  zoomButton.addEventListener("click", onZoomClick);
  deleteButton.addEventListener("click", onDeleteClick);

  const onResizePointerDown = (event: PointerEvent) => {
    if (event.button !== 0) return;
    event.preventDefault();
    event.stopPropagation();
    resizing = true;
    const startX = event.clientX;
    const startWidth = figure.getBoundingClientRect().width;
    const containerWidth = Math.min(850, figure.parentElement?.getBoundingClientRect
    resizeMove = (moveEvent: PointerEvent) => {
      const nextPx = Math.max(80, Math.min(containerWidth, startWidth + moveEvent.cli
      const percent = Math.round((nextPx / containerWidth) * 1000) / 10;
      figure.style.width = `${percent}%`;
    };
    resizeFinish = () => {
      resizing = false;
      if (resizeMove) document.removeEventListener("pointermove", resizeMove);
      if (resizeFinish) document.removeEventListener("pointerup", resizeFinish);
      resizeMove = null;
      resizeFinish = null;
      setImageAttribute(view, getPos, { width: figure.style.width || null });
    };
    document.addEventListener("pointermove", resizeMove);
    document.addEventListener("pointerup", resizeFinish, { once: true });
  };
  resizeHandle.addEventListener("pointerdown", onResizePointerDown);
  render();

  return {
    dom: figure,
    update(nextNode: ProseMirrorNode) {
      if (nextNode.type !== nodeType) return false;
      const previousSource = imageSource(attributes);
      attributes = nextNode.attrs as SafeImageAttributes;
      if (imageSource(attributes) !== previousSource) remoteLoaded = false;
      if (!resizing) render();
      return true;
    },
```

```
    stopEvent(event: Event) {
      if (event.type.startsWith("drag")) return false;
      const target = event.target as globalThis.Node;
      return toolbar.contains(target)
        || figcaption.contains(target)
        || resizeHandle.contains(target)
        || mediaWrap.classList.contains("remote-image-placeholder");
    },
    ignoreMutation() {
      return true;
    },
    destroy() {
      mediaWrap.removeEventListener("click", onClick);
      mediaWrap.removeEventListener("keydown", onKeyDown);
      captionButton.removeEventListener("click", onCaptionClick);
      figcaption.removeEventListener("blur", onCaptionBlur);
      zoomButton.removeEventListener("click", onZoomClick);
      deleteButton.removeEventListener("click", onDeleteClick);
      resizeHandle.removeEventListener("pointerdown", onResizePointerDown);
      if (resizeMove) document.removeEventListener("pointermove", resizeMove);
      if (resizeFinish) document.removeEventListener("pointerup", resizeFinish);
    },
  };
}
```

`src/styles.css` 新增拖曳把手樣式：

```
.image-drag-handle { position: absolute; top: 8px; left: 8px; width: 20px; height: 2(
.image-drag-handle::before { content: "\22EE\22EE"; letter-spacing: -1px; }
.safe-image-node:hover .image-drag-handle, .safe-image-node:focus-within .image-drag
.safe-image-node:active { cursor: grabbing; }
```

`i18n.ts` 新增 key：`image.dragHandle` → 「拖曳以移動圖片」。

驗證要點

修改後請確認：滑鼠按住圖片本體（非工具列/說明/縮放把手區域）向下拖曳，游標應變為抓取樣式並可看到 ProseMirror 的插入位置指示線；放到其他段落中間或段落之間放開，圖片應被移動而非複製或無反應；縮放把手與工具列按鈕的點擊互動不受影響（因其事件仍被 `stopEvent` 正確攔截）。